

Defensive Programming B - Birth Death Simulation

Xenopus laevis Team - Go Frogs, Go!

2026-09-15

Birth Death Process with Migration

In our first code block, we'll define a birth death simulation

```
simBirthDeathMigration <- function(
  Na, Nb, # population sizes
  b_a, b_b, # birth rates
  d_a, d_b, # death rates
  m_a2b, m_b2a, # migration rates
  t_max
){
  # run assertion checks
  stopifnot(
    Na > 0, Nb > 0,
    b_a >= 0, b_b >= 0,
    d_a >= 0, d_b >= 0,
    m_a2b >= 0, m_b2a >= 0,
    t_max > 0
  )

  # initialize time and first waiting time
  # define total event rate
  total_rate <- Na*(b_a+d_a+m_a2b) + Nb*(b_b+d_b+m_b2a)
  t <- 0
  t_w <- rexp(1, total_rate)
  t <- t + t_w

  # add initial time and sizes to storage vectors
  t_vec <- c(0)
  Na_vec <- c(Na)
  Nb_vec <- c(Nb)

  while (t < t_max){
    t_vec <- c(t_vec, t)
    # sample event type
    event <- sample(c('b_a', 'd_a', 'm_a2b', 'b_b', 'd_b', 'm_b2a'),
                  size=1,
                  prob=c(Na*b_a, Na*d_a, Na*m_a2b, Nb*b_b, Nb*d_b, Nb*m_b2a)/total_rate)
    # control flow for event type
    if (event == 'b_a'){
      Na <- Na + 1
      Na_vec <- c(Na_vec, Na)
    }
  }
```

```

    Nb_vec <- c(Nb_vec, Nb)
  }
  else if (event == 'd_a') {
    Na <- Na - 1
    Na_vec <- c(Na_vec, Na)
    Nb_vec <- c(Nb_vec, Nb)
    # check for extinction in *both* populations
    if (Na == 0 & Nb == 0){break}
  }
  else if (event == 'm_a2b') {
    Na <- Na - 1 # migrant leaving pop a
    Nb <- Nb + 1 # migrant entering pop b
    Na_vec <- c(Na_vec, Na)
    Nb_vec <- c(Nb_vec, Nb)
  }
  else if (event == 'b_a'){
    Nb <- Nb + 1
    Na_vec <- c(Na_vec, Na)
    Nb_vec <- c(Nb_vec, Nb)
  }
  else if (event == 'd_a') {
    Nb <- Nb - 1
    Na_vec <- c(Na_vec, Na)
    Nb_vec <- c(Nb_vec, Nb)
    # check for extinction in *both* populations
    if (Na == 0 & Nb == 0){break}
  }
  else { # migration from b to a
    Na <- Na + 1 # migrant entering pop a
    Nb <- Nb - 1 # migrant leaving pop b
    Na_vec <- c(Na_vec, Na)
    Nb_vec <- c(Nb_vec, Nb)
  }

  # update the total rate and draw another waiting time
  # define total event rate
  total_rate <- Na*(b_a+d_a+m_a2b) + Nb*(b_b+d_b+m_b2a)
  t_w <- rexp(1, total_rate)
  t <- t + t_w
}
df <- data.frame(t_vec, Na_vec, Nb_vec)
return(df)
}

```

Then, we'll package our simulation into a plotting function which simulates many replicates using a for loop and sets a single common seed for the simulations.

```

plotBirthDeathMigration <- function(
  Na, Nb, # population sizes
  b_a, b_b, # birth rates
  d_a, d_b, # death rates
  m_a2b, m_b2a, # migration rates
  t_max, reps, seed # max time, number of replicates, and seed
){

```

```

set.seed(seed)
p <- ggplot() + theme_bw()
for (i in 1:reps){
  df <- simBirthDeathMigration(Na, Nb, # population sizes
                               b_a, b_b, # birth rates
                               d_a, d_b, # death rates
                               m_a2b, m_b2a, # migration rates
                               t_max)

  p <- p +
    geom_line(data=df, mapping=aes(x=t_vec, y=Na_vec, alpha=0.001, color='A')) +
    geom_line(data=df, mapping=aes(x=t_vec, y=Nb_vec, alpha=0.001, color='B'))
}
p <- p +
  guides(alpha = 'none') + # remove the alpha from legend
  # change the colors and labels manually
  scale_color_manual(
    name = "Population", # Sets the legend title
    values = c("A" = "deepskyblue4", "B" = "coral1")
  ) +
  # add labels
  labs(
    x = 'Time',
    y = 'Population Size'
  )
return(p)
}

```

Finally, let's use the code to generate an example plot!

```

plot <- plotBirthDeathMigration(Na = 100, Nb = 200, # population sizes
                               b_a = 0.02, b_b = 0.03, # birth rates
                               d_a = 0.04, d_b = 0.03, # death rates
                               m_a2b = 0.03, m_b2a = 0.02, # migration rates
                               t_max = 100, reps = 25, seed=42)

plot

```

